TF4015 – Sistem Kontrol Diskrit

Praktikum

Modul 0

Pengenalan Kit Praktikum Sistem Kontrol Diskrit

Tim Pengajar TF4015

Program Studi Teknik Fisika
Fakultas Teknologi Industri
Institut Teknologi Bandung

Semester I 2026/2027

Daftar Isi

1 Pendahuluan **1**
1.1 Latar Belakang . . . 1
1.2 Tujuan Praktikum . . . 1
1.3 Peralatan dan Perangkat Lunak . . . 2
1.4 Peta Praktikum . . . 2

2 Teori Dasar **3**
2.1 Arsitektur Kit Praktikum . . . 3
2.2 Spesifikasi dan Karakteristik Komponen . . . 4
2.2.1 Aktuator: Motor DC COPAL LC32F-184VF . . . 4
2.2.2 Sensor Kecepatan: Tachometer . . . 5
2.2.3 Sensor Posisi: Potensiometer OVTER WDD35D4 . . . 6
2.2.4 Unit Pemroses: Arduino UNO . . . 7
2.2.5 Catu Daya . . . 8
2.3 Rangkaian PCB Kit Praktikum . . . 8
2.3.1 Rangkaian Driver Motor . . . 9
2.3.2 Rangkaian Pengkondisi Sinyal Sensor Kecepatan . . . 11
2.3.3 Rangkaian Pengkondisi Sinyal Sensor Posisi . . . 12
2.3.4 Tata Letak Komponen dan Konektor pada PCB . . . 12
2.4 Akuisisi Data dan Pewaktuan Diskrit . . . 13
2.4.1 Kuantisasi ADC . . . 13
2.4.2 Waktu Cacah . . . 14
2.4.3 Komunikasi Serial . . . 14
2.5 Struktur Program Praktikum . . . 15

3 Keselamatan Kerja dan SOP **16**
3.1 Keselamatan Kerja . . . 16
3.2 Pencegahan Kerusakan Kit . . . 16
3.3 Urutan Menyalakan dan Mematikan Kit . . . 17

4 Prosedur Praktikum **17**
4.1 Persiapan Sebelum Praktikum . . . 17
4.2 Percobaan 1: Identifikasi Komponen Kit . . . 18
4.3 Percobaan 2: Pemeriksaan Catu Daya . . . 18
4.4 Percobaan 3: Unggah Program dan Akuisisi Data Sensor . . . 19
4.5 Percobaan 4: Waktu Eksekusi, `delay()`, dan Waktu Cacah . . . 20
4.6 Percobaan 5: Menggerakkan Motor dan Masukan Serial . . . 20
4.7 Percobaan 6: Verifikasi Lingkungan MATLAB . . . 21

5 Laporan dan Pertanyaan **22**
5.1 Ketentuan Laporan . . . 22
5.2 Pertanyaan . . . 22

6 Penutup **23**

A Kode Program Pendukung **23**

1 Pendahuluan

1.1 Latar Belakang

Sistem Kontrol Diskrit (SKD) merupakan salah satu cabang penting dalam teknik kendali yang mempelajari analisis dan perancangan sistem kontrol dengan sinyal yang bernilai diskrit terhadap waktu. Berbeda dengan sistem kontrol analog konvensional, implementasi kontrol diskrit modern memanfaatkan perangkat komputasi digital, seperti mikrokontroler, PLC, atau komputer, sebagai pengendali utama. Pengendali digital hanya dapat "melihat" sistem fisik pada saat-saat pencacahan (*sampling*) dan hanya dapat memberikan aksi kontrol melalui konverter, sehingga pemahaman tentang perangkat keras dan pewaktuan menjadi bagian yang tidak terpisahkan dari perancangan kontrol diskrit.

Praktikum Sistem Kontrol Diskrit (**TF4015**) dirancang untuk memberikan pengalaman praktis dan aplikatif tentang pengontrolan nyata kepada mahasiswa. Melalui praktikum ini, praktikan akan dibimbing secara sistematis, mulai dari mengenali komponen fisik, mengidentifikasi karakteristik sistem seperti *deadzone* dan dinamika motor, hingga merancang, menyimulasikan, menerapkan, serta menala algoritma kontrol kalang tertutup (*closed-loop control*).

Sebelum masuk ke modul identifikasi sistem dan modul-modul kontrol, **Modul 0 – Pengenalan Kit Praktikum** ini menjadi fondasi yang krusial. Modul ini menjembatani pemahaman praktikan terhadap kit praktikum SKD yang akan digunakan secara intensif sepanjang semester. Kit praktikum ini mengintegrasikan tiga subsistem utama sistem kontrol:

1. **Sistem (*plant*)**: motor DC sebagai aktuator yang dikendalikan, dilengkapi tachometer (sensor kecepatan) dan potensiometer (sensor posisi).
2. **Kontroler**: Arduino UNO dan PCB *shield* yang berisi rangkaian driver motor serta pengkondisi sinyal sensor.
3. **Catu daya (*power supply*)**: penyedia tegangan 24 V untuk motor dan 5 V untuk rangkaian elektronik.

Pemahaman awal mengenai fungsi dan batas operasional setiap komponen, pemetaan pin antarmuka, struktur program, serta prosedur pengoperasian yang aman sangat penting, baik untuk kelancaran praktikum maupun untuk menjaga kit praktikum tetap dalam kondisi baik.

1.2 Tujuan Praktikum

Setelah menyelesaikan seluruh kegiatan pada Modul 0, praktikan diharapkan mampu:

1. Mengidentifikasi subsistem utama kit praktikum SKD (sistem, kontroler, dan catu daya) beserta level tegangan kerjanya (24 V dan 5 V).
2. Menjelaskan spesifikasi dan prinsip kerja komponen utama kit, yaitu motor DC COPAL LC32F-184VF, tachometer, potensiometer OVTER WDD35D4, Arduino UNO, serta rangkaian driver motor dan pengkondisi sinyal pada PCB.
3. Menyiapkan perangkat lunak praktikum (Arduino IDE beserta *library*, MATLAB) dan memahami struktur program praktikum.
4. Menerapkan prosedur keselamatan kerja dan prosedur standar operasional (SOP) untuk mencegah kerusakan kit praktikum.

5. Melakukan akuisisi data sensor dan menampilkannya melalui *Serial Monitor* dan *Serial Plotter*.
6. Membedakan pewaktuan program menggunakan `delay()` dan pewaktuan berbasis waktu cacah (*time sampling*).
7. Menggerakkan motor ke dua arah putar melalui masukan serial dan mengamati respons kedua sensor.

1.3 Peralatan dan Perangkat Lunak

Peralatan yang digunakan pada Modul 0 ditunjukkan pada Tabel 1.

Tabel 1: Daftar peralatan praktikum Modul 0

No	Nama Peralatan	Jumlah	Keterangan
1	Kit Praktikum SKD	1	Disediakan asisten
2	Kabel USB tipe A–B	1	Disediakan bersama kit
3	Multimeter (AVO meter)	1	Disediakan asisten
4	Laptop	1	Milik praktikan
5	Arduino IDE + library Adafruit MCP4725	1	Terinstal pada laptop
6	MATLAB (desktop atau MATLAB Online)	1	Terinstal / akun aktif
7	Perangkat lunak *spreadsheet*	1	Excel, Google Sheets, dll.
8	Program praktikum	1	Unduh dari repositori

Tautan perangkat lunak:

- Arduino IDE: https://www.arduino.cc/en/software
- MATLAB Online: https://matlab.mathworks.com
- Program praktikum: https://github.com/faqihza/TF4015---Sistem-Kontrol-Diskrit

1.4 Peta Praktikum

Modul 0 merupakan bagian pertama dari rangkaian praktikum SKD. Seluruh modul menggunakan kit praktikum yang sama dan saling berkesinambungan, seperti ditunjukkan pada Tabel 2. Keluaran setiap modul menjadi masukan bagi modul berikutnya, sehingga kit yang sama sebaiknya digunakan oleh kelompok yang sama hingga akhir praktikum.

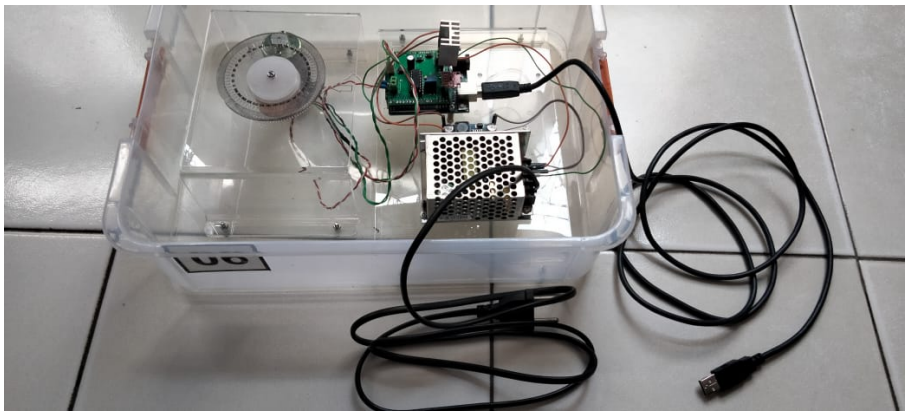
Tabel 2: Rangkaian modul praktikum Sistem Kontrol Diskrit

Modul	Judul	Keluaran utama
0	Pengenalan Kit Praktikum	Kit dan perangkat lunak siap, praktikan memahami komponen, program, dan SOP
1	Identifikasi Sistem	Kompensasi *deadzone*, waktu cacah, dan model kecepatan motor DC
2	Kontrol Kecepatan	Pengontrol P/PI/PID kecepatan hasil penalaan dan penalaan halus
3	Kontrol Posisi	Model posisi dan pengontrol posisi yang diuji terhadap gangguan
4	*Pole Placement* dan Observer	Pengontrol umpan balik keadaan dengan penempatan kutub serta observer posisi dan kecepatan

2 Teori Dasar

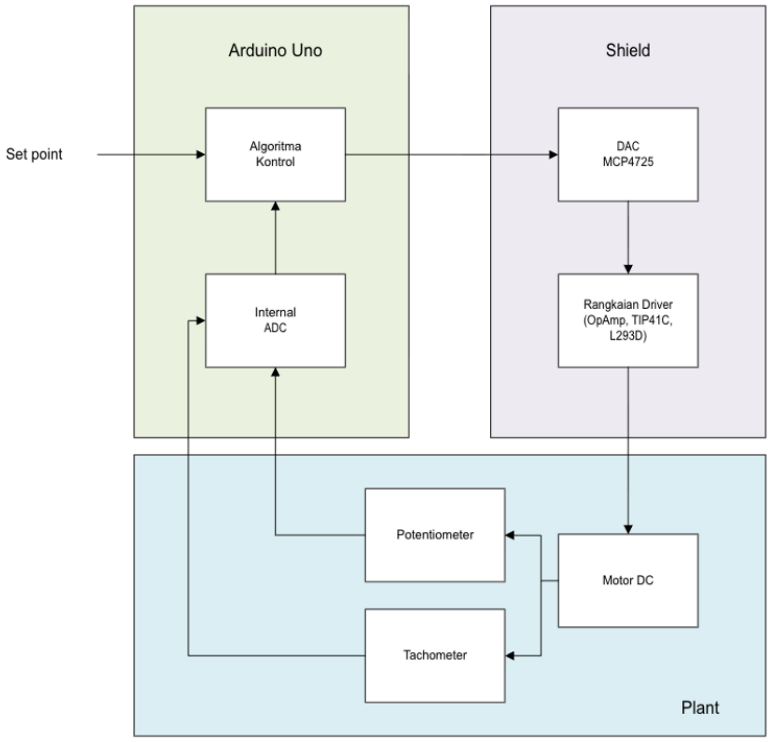
2.1 Arsitektur Kit Praktikum

Kit praktikum SKD dirancang sebagai representasi fisik dari sistem kontrol digital kalang tertutup. Foto kit praktikum ditunjukkan pada Gambar 1, sedangkan arsitekturnya ditunjukkan pada Gambar 2. Secara umum, kit dibagi menjadi tiga subsistem:



Gambar 1: Kit praktikum Sistem Kontrol Diskrit

1. **Sistem (motor dan sensor)**: objek fisik yang dikendalikan, yaitu motor DC, beserta sensor yang mengukur keluarannya, yaitu potensiometer untuk posisi dan tachometer untuk kecepatan.
2. **Kontroler (Arduino UNO dan PCB *shield*)**: Arduino UNO menjalankan program akuisisi data dan algoritma kontrol diskrit. PCB *shield* kustom dipasang di atas Arduino UNO dan berisi rangkaian driver motor, rangkaian pengkondisi sinyal sensor, serta konektor yang menghubungkan seluruh komponen kit.
3. **Catu daya**: penyedia daya listrik DC yang stabil untuk menggerakkan motor (24 V) serta menghidupkan komponen elektronik pada PCB (5 V).



Gambar 2: Arsitektur kit praktikum Sistem Kontrol Diskrit

Hubungan ketiga subsistem pada Gambar 2 merupakan struktur sistem kontrol digital yang umum. Kontroler digital berinteraksi dengan dunia fisik yang bersifat kontinu melalui dua antarmuka: *Analog-to-Digital Converter* (ADC) untuk membaca sensor dan *Digital-to-Analog Converter* (DAC) untuk memberikan aksi kontrol kepada aktuator.

2.2 Spesifikasi dan Karakteristik Komponen

2.2.1 Aktuator: Motor DC COPAL LC32F-184VF

Motor yang digunakan adalah motor DC COPAL tipe LC32F-184VF (Gambar 3), yaitu motor DC berkomutator sikat (*brushed DC motor*). Pada bagian belakang motor terpasang tachometer yang porosnya menyatu dengan poros motor. Spesifikasi motor ditunjukkan pada Tabel 3.



Gambar 3: Motor DC COPAL LC32F-184VF

Tabel 3: Spesifikasi motor DC COPAL LC32F-184VF

Parameter	**Nilai**
Tegangan kerja	24 V DC
Arus nominal (*rated*)	1,2 A
Arus maksimum	2,4 A
Torsi nominal	16,1 mNm
Torsi maksimum	48 mNm
Kecepatan nominal	6400 rpm
Kecepatan tanpa beban	8230 rpm
Torsi awal	21 mNm
Arus tanpa beban	160 mA

Motor diputar dengan memberikan beda tegangan pada kedua terminalnya (kabel merah dan kabel putih). Arah putar ditentukan oleh polaritas tegangan: membalik polaritas akan membalik arah putar. Pada kit ini, pengaturan polaritas dilakukan oleh IC driver L293D (Bagian 2.3.1), sedangkan besar tegangan menentukan torsi dan kecepatan putar motor.

2.2.2 Sensor Kecepatan: Tachometer

Tachometer (Gambar 4) terpasang menyatu dengan bagian belakang motor DC untuk mengukur kecepatan putar.

- **Tipe**: sensor kontak dan analog yang bekerja berdasarkan induksi magnetik.
- **Prinsip kerja**: poros tachometer terhubung langsung dengan poros motor. Ketika motor berputar, magnet pada poros tachometer ikut berputar dan menginduksi tegangan bolak-balik berbentuk sinusoidal pada kumparan tachometer. Sinyal keluaran dikirimkan melalui kabel berwarna hijau.

- **Karakteristik**: amplitudo (dan frekuensi) sinyal keluaran bertambah seiring bertambahnya kecepatan putar motor. Karena sinyalnya bolak-balik, sinyal ini perlu disearahkan dan difilter sebelum dapat dibaca oleh ADC Arduino (Bagian 2.3.2).



Gambar 4: Tachometer yang terpasang pada motor DC

2.2.3 Sensor Posisi: Potensiometer OVTER WDD35D4

Potensiometer putar (*rotary*) tipe WDD35D4 (Gambar 5) digunakan sebagai sensor posisi sudut. Karakteristiknya adalah sebagai berikut:



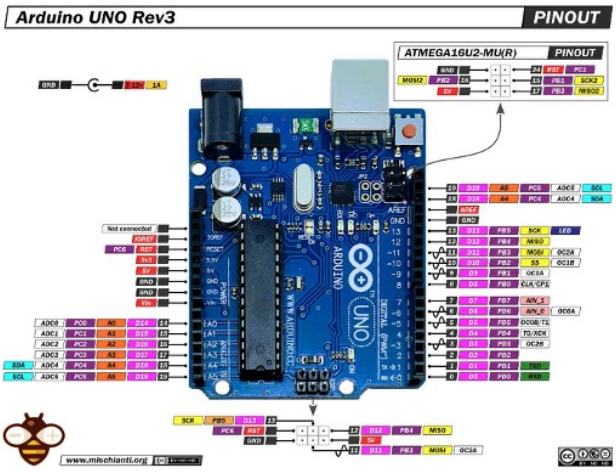
Gambar 5: Potensiometer OVTER WDD35D4

- **Hambatan**: 10 kΩ ± 15%.
- **Linearitas independen**: 0,5%, artinya penyimpangan hubungan sudut mekanis terhadap tegangan keluaran dari garis lurus tidak lebih dari 0,5% rentang penuh.
- **Sistem roda gigi**: potensiometer tidak terhubung langsung ke poros motor, melainkan digerakkan melalui sepasang roda gigi akrilik. Perbandingan jumlah gigi roda gigi motor terhadap roda gigi potensiometer adalah 16 : 100. Dengan asumsi tidak ada slip, hubungan sudut putar motor θ_m dan sudut putar potensiometer θ_p adalah
$$\theta_p = \frac{16}{100}\,\theta_m, \tag{1}$$
sehingga satu putaran penuh potensiometer setara dengan 6,25 putaran motor.
- **Pin**:

- Pin 1: *ground* (GND)
- Pin 2: sinyal tegangan keluaran (V_{out})
- Pin 3: tegangan referensi (V_{ref})

2.2.4 Unit Pemroses: Arduino UNO

Arduino UNO (Gambar 6) berfungsi sebagai kontroler digital. Mikrokontroler ATmega328P dengan *clock* 16 MHz pada Arduino UNO menjalankan pembacaan sensor, perhitungan algoritma kontrol, dan pengiriman perintah ke driver motor. PCB *shield* kit praktikum dipasang di atas Arduino UNO sehingga pin-pin Arduino langsung terhubung ke rangkaian pada PCB.



Gambar 6: Arduino UNO

- **Masukan analog**: 6 pin (A0–A5) yang terhubung ke ADC internal 10-bit. ADC mengubah tegangan 0–5 V menjadi bilangan bulat 0–1023.
- **Digital I/O**: 14 pin (D0–D13) untuk sinyal logika HIGH/LOW. Pada kit ini, pin D9–D11 digunakan untuk mengatur arah putar motor melalui L293D.
- **Komunikasi I^2C**: pin SDA (A4) dan SCL (A5) digunakan untuk mengirim perintah digital ke DAC eksternal MCP4725.
- **Komunikasi serial (USB)**: digunakan untuk mengunggah program serta mengirim dan menerima data dari laptop melalui *Serial Monitor*. Pin D0 (RX) dan D1 (TX) dipakai oleh jalur ini sehingga tidak digunakan untuk keperluan lain.

Pemetaan pin Arduino UNO pada kit praktikum dirangkum pada Tabel 4.

Tabel 4: Pemetaan pin Arduino UNO pada kit praktikum

Pin	Nama di program	Terhubung ke	Fungsi
A0	`PinTachometer`	Pengkondisi sinyal tachometer	Membaca kecepatan putar motor
A1	`PinPotensiometer`	Pengkondisi sinyal potensiometer	Membaca posisi sudut
A4 (SDA)	–	MCP4725	Data I^2C ke DAC (alamat `0x60`)
A5 (SCL)	–	MCP4725	*Clock* I^2C ke DAC
D11	`PinEn`	L293D (3-4EN)	Mengaktifkan kanal driver motor
D10	`PinIn1`	L293D (masukan arah)	HIGH saat masukan motor positif
D9	`PinIn2`	L293D (masukan arah)	HIGH saat masukan motor negatif
D0, D1	–	USB (serial)	Komunikasi dengan laptop

2.2.5 Catu Daya

Kit praktikum menggunakan dua level tegangan:

- **Power supply AC–DC 24 V** (Gambar 7): mengubah tegangan jala-jala 220 V AC menjadi 24 V DC dengan arus keluaran maksimum 1,5 A. Tegangan ini dipakai untuk menggerakkan motor melalui rangkaian driver.
- **Konverter DC–DC 5 V** (Gambar 8): menurunkan tegangan 24 V menjadi 5 V (dapat diatur melalui trimpot pada modul). Konverter ini menjadi suplai tambahan bagi IC pada PCB bila daya dari Arduino UNO (yang ditenagai port USB laptop) tidak mencukupi.



Gambar 7: Power supply 24 V



Gambar 8: Konverter DC–DC

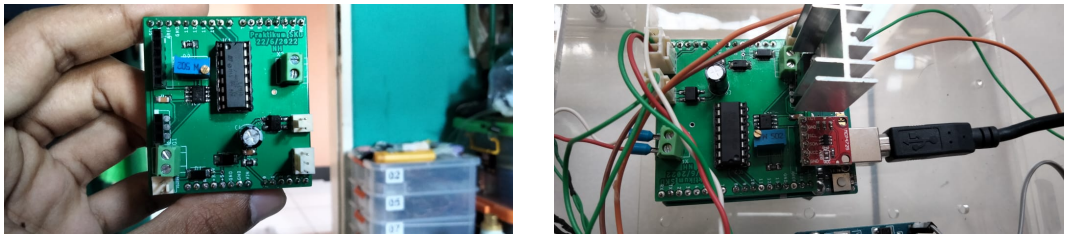
Catatan. Perhatikan bahwa arus maksimum power supply (1,5 A) lebih kecil daripada arus maksimum motor (2,4 A). Menahan poros motor yang sedang diberi masukan besar akan membebani power supply, driver, dan transistor secara berlebihan.

2.3 Rangkaian PCB Kit Praktikum

Seluruh rangkaian elektronik kit praktikum diintegrasikan pada satu PCB *shield* kustom (Gambar 9) yang terdiri atas tiga rangkaian utama:

1. rangkaian driver motor,

2. rangkaian pengkondisi sinyal sensor kecepatan, dan
3. rangkaian pengkondisi sinyal sensor posisi.

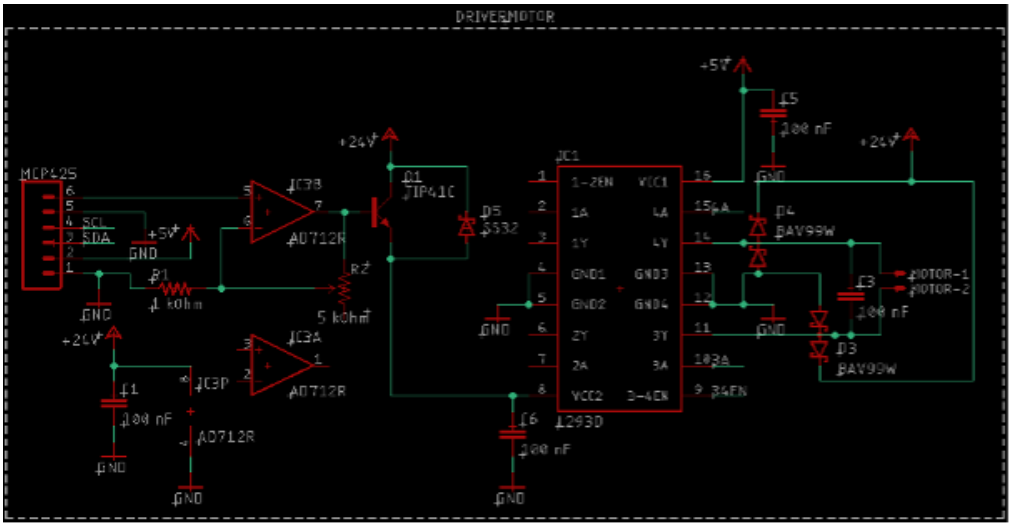


Gambar 9: PCB *shield* kit praktikum sebelum dipasang (kiri) dan setelah terpasang pada Arduino UNO beserta konektornya (kanan)

2.3.1 Rangkaian Driver Motor

Rangkaian driver motor (Gambar 10) dirancang agar Arduino UNO dapat mengatur besar tegangan motor (yang menentukan kecepatan) sekaligus arah putarnya. Rantai sinyal pada rangkaian ini adalah:

$$\text{Arduino} \xrightarrow{\text{I}^2\text{C}} \text{DAC MCP4725} \xrightarrow{0\text{–}5\,\text{V}} \text{OpAmp AD712} \rightarrow \text{Transistor TIP41C} \xrightarrow{V_{CC2}} \text{L293D} \rightarrow \text{Motor}$$



Gambar 10: Skematik rangkaian driver motor

DAC MCP4725.

- Arduino UNO tidak memiliki DAC internal; pin PWM Arduino hanya menghasilkan sinyal digital yang dimodulasi lebar pulsanya. Oleh karena itu digunakan DAC eksternal MCP4725.
- MCP4725 adalah DAC 12-bit yang berkomunikasi dengan Arduino melalui I^2C (pin SDA dan SCL). Nilai digital 0–4095 diubah menjadi tegangan analog 0–5 V pada pin OUT.

***Operational amplifier* AD712.**

- Tegangan keluaran DAC maksimum 5 V, sedangkan motor membutuhkan hingga 24 V. Tegangan DAC dikuatkan menggunakan OpAmp AD712 yang disuplai 24 V.
- OpAmp disusun sebagai penguat *non-inverting* dengan penguatan

$$A_v = 1 + \frac{R_2}{R_1}, \tag{2}$$

dengan $R_1 = 1$ kΩ dan R_2 berupa trimpot 5 kΩ. Dengan memutar trimpot, penguatan dapat diatur secara manual. Pengaturan trimpot ini akan dibahas pada Modul 1; **jangan mengubah posisi trimpot pada Modul 0**.

Transistor TIP41C.

- OpAmp hanya mampu mengalirkan arus yang kecil. Bila keluaran OpAmp langsung dihubungkan ke motor, tegangan akan turun dan motor berputar lambat.
- Transistor TIP41C dipasang sebagai penguat arus (*emitter follower*): kolektor terhubung ke suplai 24 V, basis menerima keluaran OpAmp, dan emitor menyuplai pin V_{CC2} pada L293D. Tegangan emitor kurang lebih sama dengan tegangan basis dikurangi tegangan basis–emitor (sekitar 0,7 V), tetapi dengan kemampuan arus yang jauh lebih besar.
- Transistor dilengkapi *heat sink* untuk membuang panas akibat daya yang terdisipasi.

***Half-H driver* L293D.**

- L293D berisi rangkaian *H-bridge* yang menghubungkan kedua terminal motor ke V_{CC2} atau GND sehingga polaritas tegangan motor, dan dengan demikian arah putarnya, dapat dibalik.
- L293D mendapatkan **daya motor** dari transistor TIP41C melalui pin V_{CC2}, sedangkan **sinyal logika arah** diterima langsung dari pin digital Arduino.
- Kombinasi logika pin pengendali L293D dan aksi yang dihasilkan ditunjukkan pada Tabel 5. Pada program praktikum, kombinasi ini diatur oleh fungsi `clock_wise()`, `counter_clock_wise()`, dan `fast_break()`.
- Dioda proteksi dan kapasitor *bypass* pada rangkaian berfungsi meredam tegangan balik (*back-EMF*) induktif dari motor dan derau suplai.

Catatan. Nama fungsi `clock_wise()` hanya penamaan di program. Arah putar fisik yang sebenarnya (searah atau berlawanan jarum jam) bergantung pada pemasangan kabel motor dan akan diverifikasi pada Percobaan 5.

Dengan masukan motor u dalam persen ($-100 \le u \le 100$), program mengirim nilai digital ke DAC sebesar

$$N_{DAC} = \frac{4095\,|u|}{100}, \qquad V_{DAC} = \frac{5\,|u|}{100}\ \text{V}, \tag{3}$$

sedangkan tanda u menentukan arah putar melalui L293D. Tegangan motor kemudian kurang lebih sebesar $A_v V_{DAC}$ dikurangi rugi tegangan pada transistor dan L293D, dengan batas atas mendekati tegangan suplai 24 V.

Tabel 5: Logika pengendalian arah motor oleh L293D pada program praktikum

EN (D11)	**IN1 (D10)**	**IN2 (D9)**	**Kondisi motor**	**Fungsi program**
LOW	x	x	Terputus (mengambang)	–
HIGH	HIGH	LOW	Berputar arah positif	`clock_wise()`
HIGH	LOW	HIGH	Berputar arah negatif	`counter_clock_wise()`
HIGH	LOW	LOW	Direm (kedua terminal ke GND)	`fast_break()`

2.3.2 Rangkaian Pengkondisi Sinyal Sensor Kecepatan

Sinyal tachometer berupa tegangan bolak-balik sehingga tidak dapat langsung dibaca oleh ADC Arduino yang hanya menerima tegangan 0–5 V. Rangkaian pengkondisi sinyal kecepatan (Gambar 11) terdiri atas:

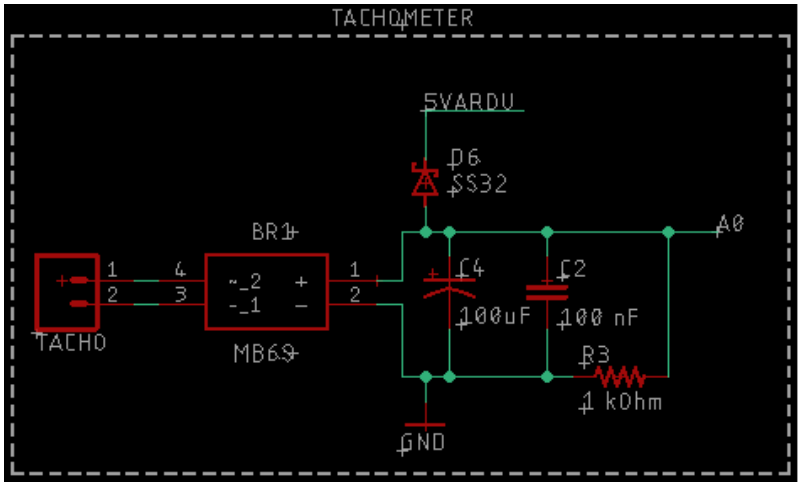
1. **Penyearah gelombang penuh** (*bridge rectifier*) MB6S (BR1), yang mengubah sinyal sinusoidal dari konektor TACHO menjadi sinyal searah (selalu positif).
2. **Filter kapasitor**, berupa kapasitor elektrolit C4 (100 μF) dan kapasitor keramik C2 (100 nF) yang dipasang paralel terhadap keluaran penyearah. C4 meratakan riak (*ripple*) hasil penyearahan, sedangkan C2 meredam derau frekuensi tinggi.
3. **Resistor beban** R3 (1 kΩ) yang dipasang paralel dengan kapasitor sebagai jalur pengosongan muatan. Tanpa R3, kapasitor akan tetap bermuatan ketika motor melambat sehingga tegangan pada pin A0 tidak mengikuti penurunan kecepatan.
4. **Dioda Schottky** D6 (SS32) yang menghubungkan keluaran ke jalur 5 V Arduino. Dioda ini menjadi penjepit (*clamp*): bila tegangan tachometer pada kecepatan tinggi melebihi 5 V, kelebihan tegangan dialirkan ke jalur 5 V sehingga pin ADC terlindungi.

Keluaran rangkaian ini dihubungkan ke pin A0 Arduino UNO dan tegangannya sebanding dengan kecepatan putar motor.

Perlu diperhatikan bahwa filter ini juga memengaruhi dinamika pengukuran. Ketika kecepatan motor turun, tegangan pada C4 berkurang melalui R3 dengan konstanta waktu

$$\tau_f = R_3\, C_4 = (1\ \text{k}\Omega)(100\ \mu\text{F}) = 0{,}1\ \text{s}. \tag{4}$$

Sebaliknya, ketika kecepatan naik, kapasitor diisi melalui penyearah dengan hambatan yang jauh lebih kecil sehingga pengisiannya berlangsung lebih cepat. Akibatnya, bacaan kecepatan dapat merespons kenaikan dan penurunan kecepatan dengan laju yang berbeda.



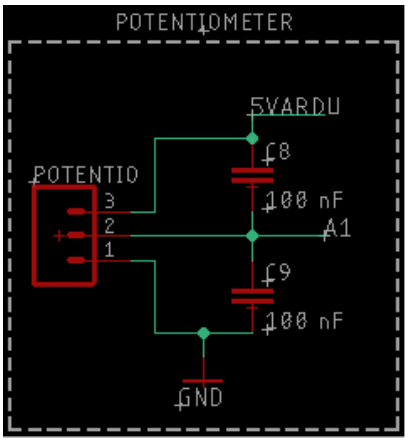
Gambar 11: Skematik rangkaian pengkondisi sinyal sensor kecepatan

2.3.3 Rangkaian Pengkondisi Sinyal Sensor Posisi

Rangkaian sensor posisi (Gambar 12) berupa rangkaian pembagi tegangan. Pin 3 potensiometer dihubungkan ke jalur 5 V Arduino (V_{ref} = 5 V) dan pin 1 ke GND, sehingga tegangan pada pin *wiper* (pin 2) sebanding dengan sudut putar potensiometer:

$$V_{out} = \frac{\theta_p}{\theta_{maks}} V_{ref}, \tag{5}$$

dengan θ_{maks} adalah sudut efektif listrik potensiometer. Dua buah kapasitor 100 nF ditambahkan sebagai filter derau: C8 antara jalur 5 V dan pin *wiper*, serta C9 antara pin *wiper* dan GND. Keduanya meredam derau frekuensi tinggi sehingga pembacaan oleh pin A1 lebih stabil. Karena V_{ref} sama dengan tegangan referensi ADC, rentang keluaran potensiometer (0–5 V) memanfaatkan rentang penuh ADC.



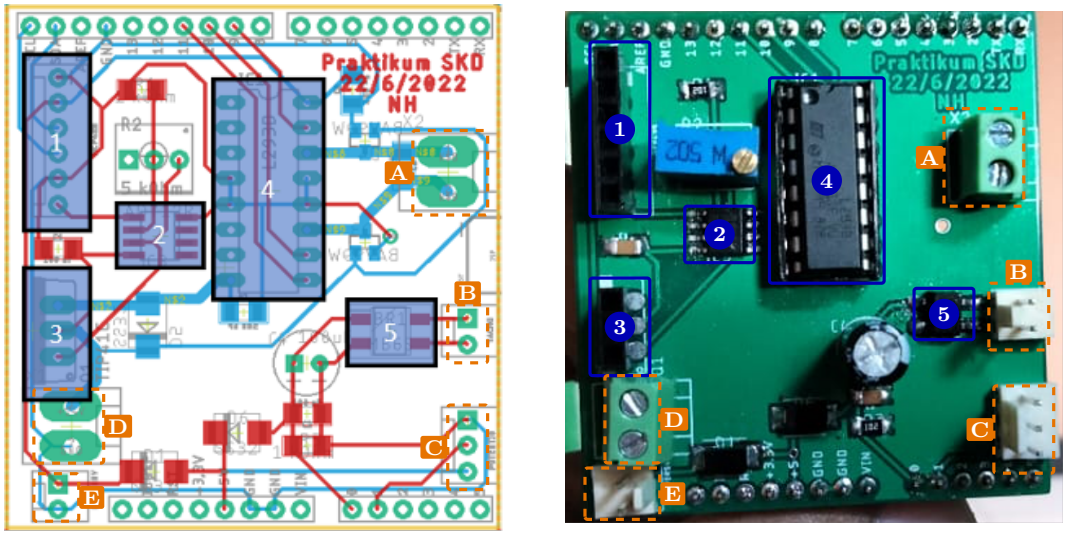
Gambar 12: Skematik rangkaian pengkondisi sinyal sensor posisi

2.3.4 Tata Letak Komponen dan Konektor pada PCB

Beberapa konektor dan komponen pada PCB dapat dilepas-pasang. Praktikan perlu mengenali posisinya karena akan digunakan pada prosedur pemeriksaan dan SOP. Gambar 13 menunjukkan tata letak PCB (kiri) dan foto PCB yang sebenarnya

(kanan). Komponen utama ditandai dengan kotak biru bernomor, sedangkan konektor ditandai dengan kotak jingga berhuruf.

- **Komponen lepasan** (dipasang pada soket sehingga dapat dicabut):
  (1) modul DAC MCP4725, dipasang pada *header* di sisi pin SDA/SCL;
  (3) transistor TIP41C (Q1), dipasang pada *header* tiga pin. Transistor sengaja dibuat dapat dilepas sehingga motor dapat dinonaktifkan dengan aman ketika program sedang diuji;
  (4) IC driver L293D (IC1), dipasang pada soket DIP 16 pin.
- **Komponen tetap** (disolder, tidak boleh dilepas):
  (2) OpAmp AD712 pada rangkaian driver motor;
  (5) penyearah MB6S (BR1) pada rangkaian pengkondisi sinyal kecepatan.
- **Konektor**:
  (A) motor DC (terminal sekrup X2),
  (B) tachometer (konektor TACHO, 2 pin),
  (C) potensiometer (konektor POTENTIO, 3 pin),
  (D) catu daya 24 V (terminal sekrup), dan
  (E) catu daya 5 V dari konverter DC–DC (konektor POWER5V, 2 pin).



Gambar 13: Posisi komponen dan konektor pada PCB *shield*: tata letak PCB (kiri) dan foto PCB (kanan). Komponen: (1) DAC MCP4725, (2) OpAmp AD712, (3) transistor TIP41C, (4) L293D, (5) penyearah MB6S. Konektor: (A) motor, (B) tachometer, (C) potensiometer, (D) catu daya 24 V, (E) catu daya 5 V

2.4 Akuisisi Data dan Pewaktuan Diskrit

2.4.1 Kuantisasi ADC

ADC 10-bit Arduino UNO dengan tegangan referensi 5 V memiliki resolusi

$$\Delta V = \frac{5\ \text{V}}{1023} \approx 4{,}89\ \text{mV per tingkat.} \tag{6}$$

Program praktikum mengubah hasil pembacaan ADC N menjadi persen agar sensor posisi dan kecepatan memiliki satuan yang seragam:

$$y_{\%} = \frac{N}{1023} \times 100\%. \tag{7}$$

Satu kali pembacaan `analogRead()` pada Arduino UNO membutuhkan waktu sekitar 0,1 ms. Program praktikum menyediakan dua cara akuisisi: pengambilan satu sampel (fungsi `sample_...`) dan perataan beberapa sampel (fungsi `smooth_...`). Perbandingan keduanya akan dianalisis pada Modul 1.

2.4.2 Waktu Cacah

Dalam kontrol diskrit, sensor dibaca dan aksi kontrol diperbarui setiap selang waktu tetap T_s yang disebut **waktu cacah** (*sampling time*). Sinyal kontinu $y(t)$ hanya diamati pada saat-saat $t = kT_s$, menghasilkan barisan

$$y[k] = y(kT_s), \qquad k = 0, 1, 2, \ldots \tag{8}$$

Agar model dan perhitungan kontrol diskrit berlaku, T_s harus konsisten dari satu cacah ke cacah berikutnya. Ada dua cara umum untuk membuat jeda pada program Arduino:

1. **Menggunakan `delay()`**. Program berhenti selama T_d, lalu menjalankan seluruh isi `loop()`. Periode satu putaran menjadi $T = T_d + T_{proses}$, dengan T_{proses} adalah waktu eksekusi instruksi (pembacaan sensor, perhitungan, komunikasi serial). Karena T_{proses} dapat berubah-ubah, periode tidak tetap. Selain itu, selama `delay()` program tidak dapat melakukan hal lain.

2. **Menggunakan `millis()` (*time sampling*)**. Program terus berputar dan memeriksa apakah waktu yang berlalu sejak cacah terakhir sudah mencapai T_s:

```
Time = millis();
if (Time - LastTime >= TimeSampling) {
  DeltaTime = Time - LastTime;
  LastTime = Time;
  // akuisisi, perhitungan kontrol, dan aksi kontrol
}
```

   Selama $T_{proses} < T_s$, periode cacah akan tetap sama dengan T_s dan program tetap dapat menjalankan tugas lain (misalnya menerima masukan serial) di luar blok `if`.

Variabel yang menyimpan nilai `millis()` dideklarasikan sebagai `unsigned long` agar tidak terjadi luapan (*overflow*) dalam waktu singkat.

2.4.3 Komunikasi Serial

Arduino UNO berkomunikasi dengan laptop melalui serial USB dengan *baudrate* **115200** bit/s pada program praktikum. Pengaturan *baudrate* pada *Serial Monitor* harus sama; bila berbeda, data yang tampil akan berupa karakter acak. Pengiriman satu karakter membutuhkan sekitar 10 bit, sehingga pada 115200 bit/s satu karakter memerlukan sekitar 0,087 ms. Mencetak banyak data setiap cacah ikut menambah T_{proses}.

Masukan dari laptop dibaca oleh fungsi `serial()` (Lampiran A, Kode 3) dengan aturan sebagai berikut:

- Teks dibaca hingga karakter *newline* (`\n`), lalu dikonversi menjadi bilangan bulat dengan `toInt()`.
- Nilai di atas 100 dibatasi menjadi 100, nilai di bawah -100 dibatasi menjadi -100.
- Pengaturan akhir baris pada *Serial Monitor* sebaiknya **Newline**. Pengaturan lain menyebabkan fungsi menunggu hingga batas waktu (*timeout*) sebelum memproses masukan.

2.5 Struktur Program Praktikum

Program praktikum disusun dalam beberapa *tab* (file `.ino`) di dalam satu folder *sketch*. Arduino IDE menggabungkan seluruh tab sesuai urutan nama file saat kompilasi, sehingga fungsi pada satu tab dapat dipanggil dari tab lain. Pembagian tab ditunjukkan pada Tabel 6.

Tabel 6: Struktur program praktikum

Tab	Isi	Modul
`SKD23.ino`	Pengaturan umum, yaitu inisialisasi serial 115200 (`general_setup()`)	0–4
`1_sensor.ino`	Definisi pin sensor, fungsi akuisisi posisi dan kecepatan (`sample`, `smooth`)	0–4
`2_actuator.ino`	Inisialisasi DAC dan pin arah, fungsi `run_motor()`	0–4
`3_serial.ino`	Fungsi `serial()` untuk membaca masukan dari *Serial Monitor*	0–4
`4_compensation.ino`	Fungsi kompensasi *deadzone*	1–4
`5_control.ino`	Fungsi perhitungan aksi kontrol	2–4
`6_state_space.ino`	Model ruang keadaan, umpan balik keadaan, dan observer	4
`9_main.ino`	Program utama: `setup()` dan `loop()`	0–4

Tab `9_main.ino` adalah satu-satunya tab yang berisi `setup()` dan `loop()`. Untuk setiap percobaan, praktikan cukup **mengganti isi tab `9_main.ino`** dengan program utama yang sesuai. Program utama contoh untuk Modul 0 ditunjukkan pada Tabel 7.

Tabel 7: Program utama contoh untuk Modul 0

No	File	Fungsi
1	`1_main_akuisisi_data`	Membaca potensiometer dan tachometer dengan tiga cara: nilai ADC mentah, persen satu sampel, dan persen rata-rata 10 sampel
2	`2_main_time`	Menambahkan waktu berjalan dan `DeltaTime` (lama satu putaran `loop()`) pada keluaran
3	`3_main_delay`	Menunjukkan pengaruh `delay()` terhadap periode putaran `loop()`
4	`4_main_time_sampling`	Menunjukkan pencacahan waktu berbasis `millis()`
5	`5_main_motor`	Menggerakkan motor menggunakan masukan dari *Serial Monitor* sambil membaca kedua sensor

Fungsi `run_motor()` (Kode 2) menerima masukan -100 hingga 100 persen. Fungsi ini membatasi masukan pada rentang tersebut, mengatur arah melalui L293D, lalu mengirim besar tegangan ke DAC sesuai Persamaan (3). Bila arah masukan berbalik dari cacah sebelumnya, fungsi ini akan mengerem motor terlebih dahulu selama satu cacah.

3 Keselamatan Kerja dan Prosedur Standar Operasional

3.1 Keselamatan Kerja

1. Pastikan tangan dan meja kerja kering. Kit praktikum terhubung dengan listrik 220 V AC melalui power supply.
2. Jangan membuka atau menyentuh terminal masukan AC pada power supply ketika kabel listrik terhubung.
3. Jauhkan jari, rambut, kabel, dan benda lain dari roda gigi ketika motor berputar.
4. Transistor TIP41C dan *heat sink*-nya dapat menjadi panas. Jangan menyentuhnya segera setelah motor dijalankan dengan masukan besar.
5. Laporkan kepada asisten apabila tercium bau hangus, terlihat asap, atau terdapat komponen yang terasa sangat panas, lalu segera cabut kabel listrik.

3.2 Pencegahan Kerusakan Kit

1. **Jangan mengubah pemasangan kabel dan komponen ketika kit sedang dialiri listrik.** Cabut kabel listrik 220 V dan kabel USB terlebih dahulu.
2. Pastikan orientasi konektor dan komponen lepasan (MCP4725, TIP41C, L293D) benar sebelum kit dinyalakan. Pemasangan terbalik dapat merusak komponen secara permanen.
3. Hindari hubung singkat. Jangan meletakkan benda logam, kabel jumper lepas, atau obeng di atas PCB.

4. **Uji program baru tanpa daya motor.** Sebelum menjalankan program yang baru diunggah atau dimodifikasi, pastikan program berjalan sesuai keinginan dengan mengamati *Serial Monitor* ketika kabel listrik 220 V belum terhubung atau ketika transistor TIP41C dilepas. Dengan demikian motor tidak akan berputar walaupun program memberikan masukan.
5. Panggil `run_motor()` hanya di dalam blok waktu cacah. Memanggil fungsi ini terus-menerus di dalam `loop()` tanpa waktu cacah akan membebani komunikasi I^2C dan memperbesar kemungkinan kerusakan kit.
6. Jangan mengubah masukan motor dari positif langsung ke negatif (atau sebaliknya). Berikan masukan nol terlebih dahulu dan tunggu motor berhenti.
7. Jangan menahan atau menghentikan poros motor dengan tangan ketika motor diberi masukan.
8. Jangan memutar trimpot penguatan OpAmp kecuali diminta dalam prosedur.

Peringatan. Kit praktikum sangat rentan rusak bila `loop()` program tidak bekerja sesuai keinginan. Selalu verifikasi program tanpa daya motor terlebih dahulu.

3.3 Urutan Menyalakan dan Mematikan Kit

Menyalakan kit:

1. Periksa kelengkapan dan pemasangan konektor serta komponen lepasan.
2. Hubungkan Arduino UNO ke laptop menggunakan kabel USB.
3. Unggah program dan verifikasi melalui *Serial Monitor* (tanpa daya motor).
4. Hubungkan kabel listrik power supply ke stop kontak 220 V. Lampu indikator power supply akan menyala.

Mematikan kit:

1. Kirim masukan nol dan tunggu hingga motor berhenti.
2. Cabut kabel listrik 220 V dari stop kontak.
3. Cabut kabel USB dari laptop.
4. Rapikan kabel dan kembalikan kit dalam kondisi lengkap.

4 Prosedur Praktikum

Prosedur Modul 0 terdiri atas persiapan dan enam percobaan. Catat seluruh hasil pengamatan pada tabel yang disediakan dan simpan data dari *Serial Monitor* ke dalam *spreadsheet* untuk dilampirkan pada laporan.

4.1 Persiapan Sebelum Praktikum

Langkah-langkah berikut dikerjakan sebelum jadwal praktikum:

1. Unduh dan instal Arduino IDE.
2. Instal *library* **Adafruit MCP4725** melalui menu *Tools* → *Manage Libraries*. *Library* pendukung yang diminta (misalnya Adafruit BusIO) ikut diinstal.

3. Unduh program praktikum dari repositori, lalu buka `SKD23.ino`. Pastikan seluruh tab pada Tabel 6 muncul.
4. Pilih *board* **Arduino Uno** pada menu *Tools* → *Board*, lalu lakukan *Verify/Compile*. Kompilasi harus berhasil tanpa galat.
5. Siapkan MATLAB, baik versi desktop maupun MATLAB Online.
6. Baca seluruh modul ini, terutama Bagian Keselamatan Kerja dan SOP.

4.2 Percobaan 1: Identifikasi Komponen Kit

Percobaan ini dilakukan **tanpa** menghubungkan kit ke listrik maupun laptop.

1. Catat nomor kit praktikum yang digunakan.
2. Identifikasi setiap komponen pada kit dengan bantuan Gambar 1 hingga Gambar 13.
3. Periksa pemasangan setiap konektor dan komponen lepasan menggunakan label pada Gambar 13, lalu lengkapi Tabel 8. Perhatikan orientasi transistor TIP41C dan takik (*notch*) IC L293D pada soketnya.
4. Putar roda gigi potensiometer secara perlahan dengan tangan dan amati bahwa roda gigi motor ikut berputar. Hitung berapa putaran poros motor untuk satu putaran potensiometer, lalu bandingkan dengan Persamaan (1).

Tabel 8: Pemeriksaan komponen kit praktikum (Nomor kit: _____)

No	Komponen / Konektor	Kondisi (✓/×)	Keterangan
1	(A) Konektor motor (X2)		
2	(B) Konektor tachometer (TACHO)		
3	(C) Konektor potensiometer (POTENTIO)		
4	(D) Konektor catu daya 24 V		
5	(E) Konektor catu daya 5 V (POWER5V)		
6	(1) Modul DAC MCP4725		
7	(3) Transistor TIP41C + heat sink		
8	(4) IC L293D		
9	Roda gigi motor dan potensiometer		
10	Kabel USB dan kabel listrik		

4.3 Percobaan 2: Pemeriksaan Catu Daya

1. Lepaskan konektor catu daya 24 V dan 5 V dari PCB.
2. Hubungkan kabel listrik power supply ke stop kontak 220 V. Pastikan lampu indikator menyala.

3. Ukur tegangan keluaran power supply menggunakan multimeter pada mode tegangan DC.
4. Ukur tegangan keluaran konverter DC–DC.
5. Cabut kabel listrik, lalu pasang kembali kedua konektor catu daya ke PCB.
6. Lengkapi Tabel 9. Laporkan kepada asisten bila tegangan menyimpang lebih dari 5% dari nilai nominal.

Tabel 9: Hasil pemeriksaan catu daya

Sumber	**Nominal (V)**	**Terukur (V)**	**Penyimpangan (%)**
Power supply AC–DC	24		
Konverter DC–DC	5		

4.4 Percobaan 3: Unggah Program dan Akuisisi Data Sensor

Percobaan ini dilakukan dengan kabel USB terhubung, tetapi **kabel listrik 220 V tidak terhubung**.

1. Hubungkan Arduino UNO ke laptop dengan kabel USB.
2. Pilih port Arduino pada menu *Tools* → *Port*. Pada Windows, port dapat dicek pada *Device Manager* (misalnya `COM3`); pada macOS/Linux, port bernama `/dev/cu.usbmodem...` atau `/dev/ttyACM...`.
3. Ganti isi tab `9_main.ino` dengan `1_main_akuisisi_data`, lalu unggah program.
4. Buka *Serial Monitor* dan atur *baudrate* menjadi 115200. Amati enam kolom data: posisi dan kecepatan dalam nilai ADC mentah, persen satu sampel, dan persen rata-rata.
5. Putar roda gigi potensiometer dengan tangan secara perlahan hingga bacaan posisi berada di sekitar 25%, 50%, dan 75%. Pada setiap posisi, catat nilai pada Tabel 10.
6. Periksa hubungan nilai ADC mentah dan nilai persen menggunakan Persamaan (7).
7. Tutup *Serial Monitor*, buka *Serial Plotter*, lalu putar potensiometer dan amati grafiknya. Simpan tangkapan layar.
8. Laporkan kepada asisten bila bacaan potensiometer tidak berubah ketika diputar atau berfluktuasi lebih dari 2% ketika diam.

Tabel 10: Hasil akuisisi data sensor posisi

Target posisi	**ADC mentah**	**Persen (sampel)**	**Persen (rata-rata)**	**Persen dari Pers. (7)**
≈25%				
≈50%				
≈75%				

4.5 Percobaan 4: Waktu Eksekusi, `delay()`, dan Waktu Cacah

Kabel listrik 220 V tetap tidak terhubung.

1. Ganti tab `9_main.ino` dengan `2_main_time`, lalu unggah. Kolom `Delta` menunjukkan lama satu putaran `loop()` dalam milidetik.
2. Salin minimal 20 baris data dari *Serial Monitor* ke *spreadsheet*, lalu hitung nilai minimum, maksimum, dan rata-rata `Delta`. Hasil ini adalah waktu proses T_{proses} program akuisisi.
3. Ganti tab `9_main.ino` dengan `3_main_delay`. Untuk nilai `TimeSampling` sebesar 10, 50, dan 100 ms (nilai `Delay` mengikuti `TimeSampling`), unggah program dan catat statistik `Delta`.
4. Ganti tab `9_main.ino` dengan `4_main_time_sampling`. Ulangi langkah 3 untuk nilai `TimeSampling` yang sama.
5. Lengkapi Tabel 11.

Tabel 11: Perbandingan periode putaran `loop()` (dalam milidetik)

Program	T_s **(ms)**	**Delta min**	**Delta maks**	**Delta rata-rata**
`2_main_time`	–			
`3_main_delay`	10 50 100			
`4_main_time_sampling`	10 50 100			

4.6 Percobaan 5: Menggerakkan Motor dan Masukan Serial

Bagian A – verifikasi program tanpa daya motor. Kabel listrik 220 V **belum** terhubung.

1. Ganti tab `9_main.ino` dengan `5_main_motor`, lalu unggah. Atur akhir baris *Serial Monitor* menjadi **Newline**.
2. Kirim masukan sesuai Tabel 12 satu per satu. Untuk setiap masukan, catat nilai kolom `Input` dan `Motor` yang tampil. Motor tidak akan berputar karena belum mendapat daya.
3. Ubah pengaturan akhir baris menjadi **No line ending**, kirim masukan `20`, lalu amati kolom `Delta` pada saat masukan diterima. Kembalikan pengaturan ke **Newline** dan kirim `0`.

Tabel 12: Pengujian masukan serial (tanpa daya motor)

No	Masukan dikirim	Kolom Input	Kolom Motor	Keterangan
1	`30`			
2	`-30`			
3	`150`			
4	`-150`			
5	`25.7`			
6	`abc`			
7	`0`			

Bagian B – menggerakkan motor. Setelah program terverifikasi:

1. Pastikan masukan terakhir adalah `0`, lalu hubungkan kabel listrik 220 V.
2. Kirim masukan `30`. Amati arah putar poros motor (dilihat dari depan, sisi roda gigi), arah perubahan bacaan potensiometer (naik atau turun), dan bacaan tachometer.
3. Kirim `0` dan tunggu motor berhenti.
4. Kirim masukan `-30` dan lakukan pengamatan yang sama.
5. Kirim `0` dan tunggu motor berhenti.
6. Ulangi untuk masukan `60` dan `-60`, dengan masukan `0` di antara keduanya.
7. Lengkapi Tabel 13. Simpan data *Serial Monitor* dan tangkapan layar *Serial Plotter* untuk satu masukan positif dan satu masukan negatif.
8. Matikan kit sesuai urutan pada SOP.

Tabel 13: Pengamatan arah putar dan respons sensor

Masukan (%)	Arah putar motor	Bacaan posisi (naik/turun)	Bacaan kecepatan (%)
30			
−30			
60			
−60			

4.7 Percobaan 6: Verifikasi Lingkungan MATLAB

1. Buka MATLAB atau MATLAB Online.
2. Unggah atau buka folder program simulasi dari repositori praktikum.
3. Jalankan program simulasi utama dan pastikan grafik respons sistem muncul tanpa galat.
4. Simpan tangkapan layar hasil simulasi. Program ini akan digunakan mulai Modul 1 dan Modul 2.

5 Laporan dan Pertanyaan

5.1 Ketentuan Laporan

Laporan Modul 0 dikumpulkan per kelompok melalui Edunex paling lambat satu minggu setelah praktikum. Laporan berupa file `.pdf` yang dilampiri *spreadsheet* data dan program utama yang dimodifikasi (bila ada), digabung dalam satu file `.zip`. Laporan minimal berisi:

1. Tabel 8 hingga Tabel 13 yang telah dilengkapi beserta tangkapan layar yang diminta.
2. Jawaban atas pertanyaan pada Bagian 5.2.
3. Tabel kontribusi anggota kelompok (Tabel 14).

Tabel 14: Kontribusi anggota kelompok

No	NIM	Nama Lengkap	Kontribusi
1			
2			
3			
4			
5			

5.2 Pertanyaan

1. Gambarkan diagram blok kit praktikum secara lebih rinci daripada Gambar 2, mulai dari masukan *Serial Monitor* hingga keluaran sensor yang diterima Arduino. Pada setiap panah, tuliskan jenis sinyal (digital/analog), rentang nilainya, dan satuannya.
2. Berapa resolusi pengukuran posisi dalam persen, dalam derajat poros potensiometer, dan dalam derajat poros motor? Gunakan Persamaan (1) dan (6), serta asumsikan rentang 0–100% setara dengan satu putaran potensiometer.
3. Berapa resolusi tegangan DAC MCP4725? Berapa perubahan masukan motor terkecil (dalam persen) yang dapat dihasilkan oleh DAC?
4. Berdasarkan Tabel 11, bandingkan periode putaran `loop()` pada program `delay()` dan program *time sampling*. Program mana yang lebih sesuai untuk sistem kontrol diskrit? Jelaskan.
5. Berdasarkan Tabel 12 dan kode fungsi `serial()`, jelaskan perlakuan program terhadap masukan di luar rentang, masukan pecahan, dan masukan bukan angka. Mengapa pengaturan *No line ending* menghasilkan `Delta` yang jauh lebih besar?
6. Jelaskan hubungan antara tanda masukan motor, arah putar motor, dan arah perubahan bacaan potensiometer berdasarkan Tabel 13.
7. Perhatikan fungsi `run_motor()`. Apa tujuan baris berikut?

   ```
   if (Direction * LastDirection < 0) Direction = fast_break();
   ```

   Apa yang dapat terjadi pada rangkaian driver bila baris ini tidak ada?

8. Perhatikan rangkaian pengkondisi sinyal tachometer (Gambar 11) dan Persamaan (4). Bandingkan konstanta waktu filter dengan waktu cacah 100 ms pada program `5_main_motor`. Bagaimana filter ini dapat memengaruhi hasil identifikasi dinamika motor yang akan dilakukan pada Modul 1?
9. Sebutkan minimal tiga langkah SOP yang paling penting menurut kelompok Anda, beserta risiko yang dicegah oleh masing-masing langkah.

6 Penutup

Dengan menyelesaikan Modul 0, kit praktikum dan perangkat lunak telah siap digunakan, dan praktikan telah memahami komponen, struktur program, serta prosedur pengoperasian kit. Pada Modul 1, praktikan akan menggunakan kemampuan ini untuk mengamati hubungan masukan dan keluaran motor, merancang kompensasi *deadzone*, menentukan waktu cacah, dan mengidentifikasi model kecepatan motor DC. Simpan program dan data Modul 0 karena akan digunakan kembali.

A Kode Program Pendukung

Kode 1: Tab `1_sensor.ino`

```
#define PinTachometer A0
#define PinPotensiometer A1

int MaxTachometer = 1023;

void sensor_setup() {
  pinMode(PinTachometer, INPUT);
  pinMode(PinPotensiometer, INPUT);
}

float sample_velocity_percentage() {
  return (float) analogRead(PinTachometer) * 100 / MaxTachometer;
}

float sample_position_percentage() {
  return (float) analogRead(PinPotensiometer) * 100 / 1023;
}

float smooth_pin(int Pin, byte SizeRead) {
  unsigned int TotalRead = 0;
  for (byte i = 0; i < SizeRead; i++) {
    TotalRead += analogRead(Pin);
  }
  return (float) TotalRead / SizeRead;
}

float smooth_velocity_percentage() {
  return smooth_pin(PinTachometer, 10) * 100 / MaxTachometer;
}

float smooth_position_percentage() {
  return smooth_pin(PinPotensiometer, 10) * 100 / 1023;
}
```

Kode 2: Tab `2_actuator.ino`

```
#include <Adafruit_MCP4725.h>

#define PinEn 11
#define PinIn1 10
#define PinIn2 9

int Direction, LastDirection = 0;
Adafruit_MCP4725 DAC;

void actuator_setup() {
  DAC.begin(0x60);
  pinMode(PinEn, OUTPUT);
  pinMode(PinIn1, OUTPUT);
  pinMode(PinIn2, OUTPUT);
  digitalWrite(PinEn, HIGH);
  digitalWrite(PinIn1, LOW);
  digitalWrite(PinIn2, LOW);
}

int clock_wise() {
  digitalWrite(PinIn1, HIGH);
  digitalWrite(PinIn2, LOW);
  return 1;
}

int counter_clock_wise() {
  digitalWrite(PinIn2, HIGH);
  digitalWrite(PinIn1, LOW);
  return -1;
}

int fast_break() {
  digitalWrite(PinIn1, LOW);
  digitalWrite(PinIn2, LOW);
  return 0;
}

float percentage_limit(float Percent) {
  if (Percent > 100.0) return 100.0;
  else if (Percent < -100.0) return -100.0;
  else return Percent;
}

void set_voltage(int Sign, float Percent) {
  uint32_t InputMCP = Sign * 4095 * Percent / 100;
  DAC.setVoltage(InputMCP, false);
}

float run_motor(float InputPercentage) {
  InputPercentage = percentage_limit(InputPercentage);
  if (InputPercentage > 0.0) Direction = clock_wise();
  else if (InputPercentage < 0.0) Direction = counter_clock_wise();
  else Direction = fast_break();
  if (Direction * LastDirection < 0) Direction = fast_break();
  set_voltage(Direction, InputPercentage);
  LastDirection = Direction;
  return (float) Direction * Direction * InputPercentage;
}
```

Kode 3: Tab `3_serial.ino`

```
int serial(int SerialRead) {
  if (Serial.available() > 0) {
    String StringSerial = Serial.readStringUntil('\n');
    SerialRead = StringSerial.toInt();
    if (SerialRead > 100) return 100;
    else if (SerialRead < -100) return -100;
  }
  return SerialRead;
}
```

Kode 4: Program utama `5_main_motor`

```
unsigned long Time, StartTime, LastTime;
unsigned int TimeSampling = 100, DeltaTime;
int InputSerial = 0;
float Motor = 0.0;
float Poten1, Poten2, Tacho1, Tacho2;

void setup() {
  general_setup();
  sensor_setup();
  actuator_setup();
  print_header();
  StartTime = millis();
  LastTime = StartTime;
}

void loop() {
  InputSerial = serial(InputSerial);
  Time = millis();
  if (Time - LastTime >= TimeSampling) {
    DeltaTime = Time - LastTime;
    LastTime = Time;
    Motor = run_motor((float) InputSerial);
    Poten1 = sample_position_percentage();
    Poten2 = smooth_position_percentage();
    Tacho1 = sample_velocity_percentage();
    Tacho2 = smooth_velocity_percentage();
    Serial.print(Time - StartTime - TimeSampling);
    Serial.print("\t");
    Serial.print(DeltaTime);
    Serial.print("\t");
    Serial.print(InputSerial);
    Serial.print("\t");
    Serial.print(Motor);
    Serial.print("\t");
    Serial.print(Poten1);
    Serial.print("\t");
    Serial.print(Poten2);
    Serial.print("\t");
    Serial.print(Tacho1);
    Serial.print("\t");
    Serial.println(Tacho2);
  }
}

void print_header() {
  Serial.println("Time\tDelta\tInput\tMotor\tPos 1\tPos 2\tKec 1\tKec 2");
}
```